Universidad Manuela Beltrán
Acreditada en Alta Calidad Multicampus

Diseño de una aplicación sistemática que gestiona parqueaderos

Presentado por:

- Bryan Failach
- Alejandro Castellanos
- Brayan Bello

Grupo 2

Bogotá, Noviembre 4 2026

Diseño de una aplicación sistemática que gestiona parqueaderos

Entregable a:

Ing. Fernando B.

Grupo 2

Bogotá, Noviembre 4 2026

Índice

3. Abstract 4
4. Introducción 5
5. Justificación 6
6. Problemática 6
7. Necesidad 7
8. Pregunta problema 7
9. Requerimientos funcionales 8
10. Requerimientos no funcionales 9
11. Marco conceptual 10
    11.1 Conceptos fundamentales del proyecto 10
    11.2 Conceptos de sistemas transaccionales 11
    11.3 Conceptos tecnológicos 12
    11.4 Aplicación al proyecto 13
12. Marco contextual 14
13. Marco integrador: incorporación del servicio de Inteligencia Artificial 16
14. Modelado de una red neuronal simple 18
    14.1 Propósito 18
    14.2 Entradas y salida 18
    14.3 Arquitectura de la red 19
    14.4 Apoyo al proceso transaccional 20
15. Estado del arte 21
    15.1 Antecedentes internacionales 21
    15.2 Antecedentes en Colombia y Bogotá 22
    15.3 Aportes para Parkway 22
16. Presupuesto 23
    16.1 Recurso humano y tiempo 23
    16.2 Recursos tecnológicos, software y otros costos 24
17. Cronograma 25
18. Referencias 28

3. Abstract

En muchos parqueaderos los usuarios llegan sin saber cuántos espacios hay disponibles, y la administración debe registrar a mano las entradas, las salidas y la ocupación. Este proyecto propone un sistema de gestión que centraliza esa información y la mantiene actualizada en tiempo real. Con él se espera lograr una mayor disponibilidad del sistema, reducir los errores de registro y evitar que un mismo espacio se asigne a más de un vehículo. Su arquitectura, además, permite ampliar la capacidad a medida que aumente el número de usuarios o de parqueaderos. La propuesta beneficia a dos grupos. Los usuarios pueden consultar los espacios libres y hacer los procesos de entrada y salida con más agilidad. Los administradores cuentan con una herramienta para controlar los espacios, los vehículos y el funcionamiento general del parqueadero.

Palabras clave: gestión de parqueaderos, disponibilidad en tiempo real, control de acceso vehicular, escalabilidad.

4. Introducción

La gestión de un parqueadero depende de saber en todo momento qué espacios están ocupados y cuáles están libres. En muchos casos ese control se hace a mano: el personal anota las entradas y las salidas, y el usuario solo se entera de si hay cupo cuando ya llegó. Este método es lento, favorece los errores de registro y puede llevar a que un mismo espacio se asigne a dos vehículos.

Este proyecto propone un sistema para administrar la disponibilidad de espacios de forma práctica y eficiente. Con él, los usuarios pueden consultar los cupos disponibles y los administradores pueden controlar los vehículos y la ocupación desde una misma herramienta.

El sistema se basa en una arquitectura distribuida. En lugar de depender de un único servidor, varios nodos se comunican entre sí y comparten la información sobre los espacios. Si uno de ellos falla, los demás siguen respondiendo a las consultas y el servicio no se interrumpe. Este diseño también permite agregar nodos cuando aumenta el número de usuarios o de parqueaderos.

5. Justificación

Este sistema parte de la problemática de la administración de algunos parqueaderos, que manejan una gestión deficiente de entradas, salidas y disponibilidad. Por consiguiente, el usuario final no conoce estos datos, indispensables para decidir si parquear o no su vehículo. También, este sistema contribuye al correcto funcionamiento del parqueadero, ya que no tendrá que lidiar con la situación que ocurre cuando un vehículo ingresa pero no hay disponibilidad.

6. Problemática

En algunos parqueaderos, la gestión de los espacios disponibles se realiza de manera manual o mediante sistemas que no actualizan la información en tiempo real. Esto genera dificultades tanto para los usuarios como para los administradores. Los usuarios pueden llegar al parqueadero sin conocer previamente la disponibilidad de espacios, lo que aumenta el tiempo de búsqueda y puede ocasionar congestión en los accesos.

Por otra parte, los administradores deben controlar manualmente las entradas, salidas y ocupación de los espacios lo que aumenta la posibilidad de los errores en los registros. Una actualización incorrecta puede provocar que un espacio libre aparezca como ocupado o que un mismo espacio sea asignado a más de un vehículo. Esta situación pone en evidencia la necesidad de contar con un sistema que permita gestionar y consultar la disponibilidad de los espacios de manera centralizada, actualizada y confiable, así va a facilitar el acceso de los usuarios así como las labores administrativas.

7. Necesidad

Existe la necesidad de implementar un sistema distribuido para la gestión de parqueaderos que permita conocer y actualizar en tiempo real el estado de los espacios disponibles, ocupados y asignados. El sistema debe facilitar a los usuarios la consulta de disponibilidad antes y durante su llegada al parqueadero, reduciendo el tiempo destinado a buscar un espacio.

Asimismo, los administradores necesitan una herramienta que les permita controlar de manera eficiente las entradas, salidas, vehículos y espacios disponibles, disminuyendo los errores de registro y evitando asignaciones duplicadas.

Además, se requiere que el sistema pueda mantener su funcionamiento ante posibles fallas de alguno de sus nodos y que se puedan incorporar nuevos nodos cuando aumente la cantidad de usuarios o parqueaderos. Esto busca mejorar la disponibilidad del servicio, agilizar la gestión de los espacios y ofrecer una solución que pueda crecer de acuerdo con las necesidades del sistema.

8. Pregunta problema

¿De qué manera el diseño e implementación de un sistema distribuido optimiza la gestión y disponibilidad de espacios de parqueadero en tiempo real para reducir el tiempo de búsqueda de los usuarios ?

9. Requerimientos funcionales

Los requerimientos funcionales describen las acciones que el sistema Parkway permite realizar a cada actor. Se derivan de la problemática y de la necesidad planteadas en los numerales 6 y 7, y corresponden a las tablas definidas en el modelo relacional del repositorio del proyecto (bfailach/Parkway). El sistema reconoce tres actores: el cliente, que consulta cupos y reserva, el operador, que registra entradas, salidas y pagos en la portería, y el administrador, que configura sedes, espacios, tarifas y usuarios y consulta los reportes.

Tabla 1. ***Requerimientos funcionales del sistema Parkway***

Código	Nombre	Descripción de la funcionalidad	Actor	Resultado esperado
RF01	Iniciar sesión	El sistema valida correo y contraseña, emite un token JWT y habilita las opciones del rol del usuario.	Cliente, operador, administrador	Acceso a las funciones del rol. Los intentos fallidos quedan rechazados.
RF02	Gestionar usuarios	El administrador crea, consulta, actualiza e inactiva usuarios con rol CLIENTE, OPERADOR o ADMINISTRADOR y turno asignado.	Administrador	Usuario registrado con correo y documento únicos.
RF03	Gestionar parqueaderos y espacios	El administrador registra sedes con su capacidad y crea espacios por tipo de vehículo. También marca espacios fuera de servicio.	Administrador	Sedes y espacios disponibles para asignación, con código único por sede.
RF04	Registrar vehículos	El sistema registra la placa, el tipo, la marca y el color del vehículo, y lo asocia a un cliente cuando corresponde.	Operador, cliente	Vehículo identificado por placa única.
RF05	Consultar disponibilidad	El usuario consulta los espacios libres por sede y tipo de vehículo antes y durante su llegada.	Cliente, operador	Número y lista de espacios libres actualizados al momento de la consulta.
RF06	Registrar ingreso	El operador registra la placa. El sistema asigna un espacio libre del tipo correcto, abre el registro y marca el espacio como OCUPADO en una sola transacción.	Operador	Registro ABIERTO con hora de ingreso. Ningún espacio queda asignado a dos vehículos.
RF07	Registrar salida	El operador registra la salida. El sistema calcula los minutos de permanencia y el valor según la tarifa activa, cierra el registro y libera el espacio.	Operador	Registro CERRADO con valor calculado y espacio en estado LIBRE.
RF08	Procesar pago y emitir comprobante	El operador registra el pago en efectivo, tarjeta o transferencia. El sistema genera un comprobante con número único.	Operador	Pago APROBADO y comprobante emitido. Un mismo pago externo no se registra dos veces.
RF09	Anular pago	El administrador anula un pago con error y el sistema conserva el registro original con estado ANULADO.	Administrador	Pago anulado con trazabilidad completa.

Código	Nombre	Descripción de la funcionalidad	Actor	Resultado esperado
RF10	Gestionar reservas	El cliente reserva un espacio para una franja horaria, la consulta o la cancela. El sistema expira las reservas no cumplidas.	Cliente	Espacio en estado RESERVADO durante la franja solicitada.
RF11	Gestionar tarifas	El administrador define valor por hora, fracción y día para cada tipo de vehículo, con una sola tarifa activa por tipo.	Administrador	Tarifa vigente aplicada a los cobros nuevos.
RF12	Actualizar disponibilidad en tiempo real	El sistema envía por WebSocket cada cambio de estado de un espacio a las pantallas abiertas.	Sistema	Pantallas actualizadas sin recargar la página.
RF13	Generar reportes	El administrador genera reportes de ocupación por franja horaria, rotación por espacio e ingresos por periodo y sede.	Administrador	Reporte en pantalla y exportable a CSV.
RF14	Consultar bitácora de transacciones	El administrador consulta las operaciones registradas por usuario, tipo, entidad, estado y fecha.	Administrador	Listado de transacciones CONFIRMADAS y REVERTIDAS.
RF15	Consultar predicción de ocupación	El usuario consulta la ocupación estimada de una sede para las próximas horas, calculada por el servicio de IA (numeral 13).	Cliente, administrador	Porcentaje de ocupación esperado por hora con su rango de confianza.

Fuente: elaboración propia.

10. Requerimientos no funcionales

Los requerimientos no funcionales fijan las condiciones de calidad, seguridad y operación del sistema. Cada uno incluye un criterio de verificación para comprobar su cumplimiento durante las pruebas del prototipo. La clasificación sigue las características de calidad del modelo ISO/IEC 25010 (International Organization for Standardization [ISO], 2023).

Tabla 2. ***Requerimientos no funcionales del sistema Parkway***

Código	Atributo	Requerimiento	Criterio de verificación
RNF01	Seguridad	El sistema exige autenticación con token JWT en todas las rutas protegidas y restringe cada función según el rol. Las contraseñas se almacenan con hash, nunca en texto plano.	Una petición sin token o con rol no autorizado recibe respuesta 401 o 403 en el 100 % de los casos de prueba. El token expira a las 8 horas.
RNF02	Rendimiento	La consulta de disponibilidad y el registro de ingreso responden en tiempos aptos para la operación en portería.	Con 100 usuarios concurrentes, el 95 % de las consultas responde en menos de 2 segundos y el 95 % de los ingresos en menos de 3 segundos.
RNF03	Integridad de los datos	Cada operación de ingreso, salida o pago se ejecuta como una transacción ACID. La base de datos impide dos registros abiertos para la misma placa o el mismo espacio.	En una prueba de 50 ingresos simultáneos sobre 10 espacios, el sistema registra 10 ingresos y rechaza 40, sin espacios duplicados.

Código	Atributo	Requerimiento	Criterio de verificación
RNF04	Disponibilidad	El servicio opera durante el horario del parqueadero y sigue respondiendo si falla uno de sus nodos de aplicación.	Disponibilidad mensual igual o superior al 99 %. Al detener un nodo, otro atiende las peticiones en menos de 30 segundos.
RNF05	Usabilidad	La interfaz del operador permite registrar un ingreso o una salida con pocos pasos y mensajes claros.	Un operador nuevo registra un ingreso en menos de 30 segundos y en máximo 3 acciones. 4 de 5 usuarios de prueba completan la tarea sin ayuda.
RNF06	Compatibilidad	La aplicación web funciona en los navegadores y dispositivos de uso común en portería y en teléfonos de clientes.	Funcionamiento correcto en las dos últimas versiones de Chrome, Firefox y Edge, y en pantallas desde 360 px de ancho.
RNF07	Mantenibilidad	El código sigue la arquitectura en tres capas, usa TypeScript y separa rutas, controladores, servicios y repositorios.	Cobertura de pruebas unitarias igual o superior al 60 % en la capa de servicios. Cada módulo documentado en el README.
RNF08	Trazabilidad	El gestor de transacciones registra cada operación con usuario, tipo, entidad, estado y fecha en la tabla transaccion.	El 100 % de las operaciones de ingreso, salida, pago y anulación aparece en la bitácora, incluidas las revertidas.

Fuente: elaboración propia.

11. Marco conceptual

Este marco reúne los conceptos que sustentan el diseño de Parkway. Se organiza en cuatro partes: los conceptos propios de la gestión de parqueaderos, los conceptos de sistemas transaccionales, los fundamentos de las tecnologías elegidas y la forma en que cada concepto se aplica en el prototipo.

11.1 Conceptos fundamentales del proyecto

Gestión de parqueaderos. Es el conjunto de procesos que controla el uso de los espacios de estacionamiento: el ingreso y la salida de vehículos, la asignación de espacios, el cobro y la información al usuario. En Parkway estos procesos dejan de depender de planillas y pasan a una base de datos central.

Disponibilidad y ocupación. La ocupación es la proporción de espacios en uso frente a la capacidad total de una sede en un momento dado. La disponibilidad es el número de espacios libres. Ambos datos cambian con cada ingreso y cada salida, por lo cual el sistema los recalcula en cada transacción.

Rotación. Indica cuántos vehículos usan un mismo espacio durante un periodo. Un valor alto refleja estancias cortas y un uso intensivo del espacio. El reporte de rotación (RF13) ayuda al administrador a decidir tarifas y horarios.

Búsqueda de estacionamiento. Shoup (2006) define el *cruising for parking* como el tiempo que un conductor circula en busca de un espacio libre. Este tiempo genera congestión, consumo de combustible y demoras. Shoup (2005) relaciona esta búsqueda con la falta de información sobre la oferta de espacios. Parkway ataca este problema con la consulta previa de cupos (RF05) y la predicción de ocupación (RF15).

11.2 Conceptos de sistemas transaccionales

Sistema de procesamiento de transacciones. Laudon y Laudon (2020) lo describen como el sistema que registra las operaciones rutinarias de una organización, como ventas, pagos o movimientos de inventario, y sirve de base para los demás sistemas de información. En Parkway, cada ingreso, salida, pago y reserva es una operación de este tipo.

Transacción y propiedades ACID. Una transacción es una unidad de trabajo que ejecuta varias operaciones sobre la base de datos como un todo. Gray y Reuter (1992) y Silberschatz et al. (2020) la caracterizan con cuatro propiedades. La atomicidad garantiza que se ejecuten todas las operaciones o ninguna. La consistencia mantiene las reglas de integridad. El aislamiento evita que transacciones concurrentes interfieran entre sí. La durabilidad conserva los cambios confirmados aun ante fallas.

Operaciones CRUD. Las operaciones de crear, leer, actualizar y eliminar componen la gestión básica de registros (Elmasri y Navathe, 2016). En Parkway corresponden a la gestión de usuarios, sedes, espacios, vehículos y tarifas (RF02 a RF04 y RF11).

Control de concurrencia. Cuando dos operadores intentan asignar el mismo espacio al mismo tiempo, el sistema debe permitir solo una asignación. PostgreSQL ofrece niveles de aislamiento, bloqueos de filas e índices únicos para resolver estos conflictos (The PostgreSQL Global Development Group, 2024a). El control optimista usa un número de versión en cada fila y rechaza la actualización si otra transacción modificó la fila primero (Elmasri y Navathe, 2016).

Integridad referencial y restricciones. Las claves primarias, las claves foráneas y las restricciones CHECK impiden datos inválidos, como un registro de estacionamiento sin vehículo o un monto negativo. Los índices únicos parciales aplican una regla de unicidad solo a las filas que cumplen una condición (The PostgreSQL Global Development Group, 2024b).

Idempotencia. Una operación es idempotente cuando ejecutarla varias veces produce el mismo resultado que ejecutarla una vez. En pagos electrónicos, una clave de idempotencia evita cobrar dos veces cuando la pasarela reenvía una confirmación.

Bitácora de transacciones. Es el registro de cada operación con su autor, su resultado y su fecha. Permite auditar el sistema y reconstruir lo ocurrido ante un error (Gray y Reuter, 1992).

11.3 Conceptos tecnológicos

Arquitectura en tres capas. Separa la presentación, la lógica de negocio y los datos. Cada capa se modifica sin alterar las demás, lo que favorece la mantenibilidad (Sommerville, 2016). Parkway replica la capa de negocio en varios nodos sobre una base de datos común, lo que responde a la arquitectura distribuida descrita en la introducción.

PostgreSQL. Es un sistema gestor de bases de datos relacional de código abierto. Soporta transacciones ACID, control de concurrencia multiversión (MVCC), índices parciales y tipos JSONB (The PostgreSQL Global Development Group, 2024a).

Node.js, Express y TypeScript. Node.js ejecuta JavaScript en el servidor con un modelo de eventos no bloqueante, adecuado para atender muchas peticiones cortas (OpenJS Foundation, s. f.). Express organiza las rutas de la API. TypeScript añade tipos estáticos que detectan errores antes de la ejecución (Microsoft, s. f.).

React. Es una biblioteca de JavaScript para construir interfaces a partir de componentes reutilizables (Meta Platforms, s. f.). Parkway la usa para las vistas de cliente, operador y administrador.

API REST. Fielding (2000) define REST como un estilo de arquitectura basado en recursos identificados por URL y operaciones HTTP sin estado. La API de Parkway expone recursos como /api/estacionamiento/disponibilidad.

WebSocket. Es un protocolo que mantiene un canal bidireccional entre el navegador y el servidor (Fette y Melnikov, 2011). El servidor envía los cambios de estado de los espacios sin que el cliente recargue la página (RF12).

JSON Web Token (JWT). Es un formato compacto y firmado para transmitir la identidad y el rol de un usuario entre el cliente y el servidor (Jones et al., 2015). Parkway lo usa para la autenticación y el control de acceso (RF01, RNF01).

11.4 Aplicación al proyecto

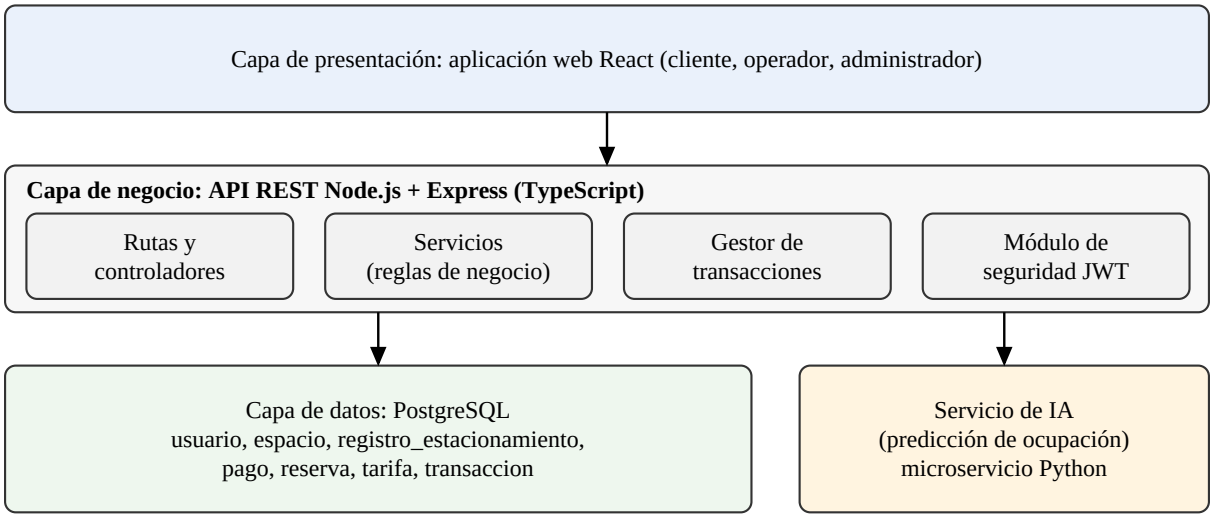
La Tabla 3 relaciona cada concepto con su aplicación concreta en el prototipo y con los requerimientos que lo implementan.

Tabla 3. ***Relación entre conceptos y funcionalidades del prototipo***

Concepto	Aplicación en Parkway	Requerimientos
Transacción ACID	El gestor de transacciones abre BEGIN, ejecuta asignación de espacio y apertura del registro, y confirma con COMMIT. Si una operación falla, ejecuta ROLLBACK y el espacio no cambia de estado.	RF06, RF07, RF08, RNF03
Control de concurrencia	Los índices únicos parciales ux_registro_abierto_por_placa y ux_registro_abierto_por_espacio impiden dos registros abiertos para la misma placa o el mismo espacio. La columna version de espacio aplica control optimista en cambios administrativos.	RF03, RF06, RNF03
Integridad referencial	Las claves foráneas enlazan registro, vehículo, espacio, tarifa y operador. Las restricciones CHECK validan estados, montos y fechas.	RF04, RF07, RF11
Regla de negocio única	El índice ux_tarifa_activa_por_tipo garantiza una sola tarifa activa por tipo de vehículo.	RF11
Idempotencia	La columna referencia_externa de pago es única y evita registrar dos veces el mismo pago de la pasarela.	RF08
Bitácora	La tabla transaccion guarda cada operación con estado CONFIRMADA o REVERTIDA y el detalle del error en JSONB.	RF14, RNF08
Tres capas y REST	Rutas, controladores, servicios y repositorios separan la lógica. La interfaz React consume la API REST.	RNF07
WebSocket	Cada COMMIT que cambia el estado de un espacio emite un evento a las pantallas conectadas.	RF12
JWT y roles	El middleware verificarToken valida el token y el rol antes de cada operación protegida.	RF01, RNF01

Fuente: elaboración propia.

Figura 1. ***Arquitectura del sistema Parkway***



Fuente: elaboración propia.

12. Marco contextual

Contexto del proyecto. El proyecto surge en Bogotá, en parqueaderos privados pequeños y medianos de uso público, como los que atienden zonas comerciales, universidades y edificios de oficinas. Muchos de estos establecimientos operan con una o dos personas en portería, una planilla de papel o una hoja de cálculo, y un tablero manual que indica si hay cupo. El conductor no conoce la disponibilidad hasta que llega a la entrada.

Problemática identificada. Como se expuso en el numeral 6, el control manual produce tres fallas. El usuario llega sin saber si hay cupo y prolonga su búsqueda. El personal comete errores de registro que muestran espacios libres como ocupados o al revés. Y un mismo espacio llega a asignarse a dos vehículos. A esto se suma la falta de datos históricos para planear tarifas, turnos y capacidad.

Usuarios o beneficiarios. La Tabla 4 resume los actores del sistema y sus necesidades.

Tabla 4. ***Usuarios del sistema y necesidades***

Actor	Descripción	Necesidades principales
Cliente	Conductor que usa el parqueadero de forma ocasional o frecuente.	Conocer los cupos libres antes de llegar, reservar un espacio, conocer el valor a pagar y saber a qué hora hay menos ocupación.
Operador	Empleado de portería que atiende entradas, salidas y cobros en un turno.	Registrar ingresos y salidas en segundos, calcular el cobro sin errores y emitir el comprobante.

Actor	Descripción	Necesidades principales
Administrador	Propietario o encargado de una o varias sedes.	Configurar espacios y tarifas, controlar la ocupación, revisar pagos y anulaciones, y planear con base en reportes y predicciones.

Fuente: elaboración propia.

Necesidad que se pretende atender. El parqueadero necesita un registro único y confiable del estado de cada espacio, actualizado con cada operación y visible para clientes, operadores y administradores. La necesidad también incluye operar sin interrupciones ante la falla de un nodo y crecer a nuevas sedes sin rediseñar el sistema.

Descripción del producto o prototipo. Parkway es un sistema transaccional web para la gestión de parqueaderos. Su propósito es controlar en tiempo real la ocupación de los espacios y registrar cada ingreso, salida, pago y reserva como una transacción verificable. El sistema gestiona información de usuarios y roles, sedes, espacios, tipos de vehículo, vehículos, tarifas, reservas, registros de estacionamiento, pagos, comprobantes y la bitácora de transacciones. Sus funciones principales son la consulta de disponibilidad, el registro de ingreso con asignación automática, el registro de salida con cálculo de tarifa, el procesamiento de pagos, la gestión de reservas, los reportes y la predicción de ocupación.

El flujo de uso es el siguiente. El cliente consulta desde su teléfono los cupos de la sede y, si lo desea, reserva. Al llegar, el operador digita la placa y el sistema asigna un espacio libre. Al salir, el operador registra la salida, el sistema calcula el valor y el operador registra el pago. Cada cambio aparece de inmediato en las pantallas abiertas. El administrador revisa los reportes y la predicción para ajustar tarifas y turnos.

Alcance del proyecto. La Tabla 5 precisa qué incluye esta etapa académica y qué queda fuera.

Tabla 5. ***Alcance del proyecto en la etapa académica***

Incluido en el alcance	Fuera del alcance
Base de datos PostgreSQL con tablas, claves, restricciones y datos de prueba.	Integración con hardware: barreras, sensores de piso o cámaras.
API REST en Node.js y TypeScript con gestor de transacciones y bitácora.	Conexión real con una pasarela de pagos. Los pagos electrónicos se simulan.
Operaciones de RF01 a RF14 y la interfaz web en React para los tres roles.	Aplicación móvil nativa para Android o iOS.
Actualización en tiempo real por WebSocket.	Facturación electrónica ante la DIAN.
Diseño conceptual del servicio de IA (RF15) documentado en el numeral 13.	Implementación y despliegue en producción del servicio de IA en esta entrega.

Incluido en el alcance	Fuera del alcance
Pruebas en entorno local con datos de prueba de una o dos sedes.	Despliegue en la nube con múltiples nodos en producción.

Fuente: elaboración propia.

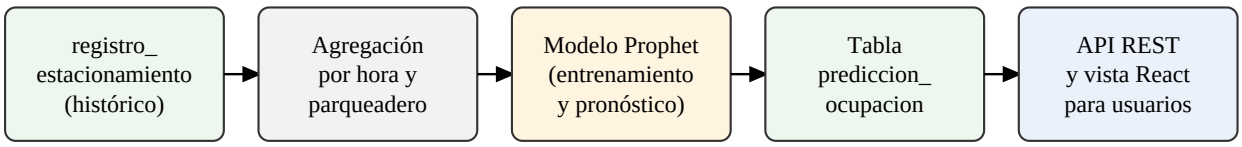
13. Marco integrador: incorporación del servicio de Inteligencia Artificial

Servicio de Inteligencia Artificial propuesto. Se propone incorporar un servicio de predicción de ocupación basado en Prophet, una biblioteca de código abierto para pronóstico de series de tiempo publicada por Meta (Taylor y Letham, 2018). Prophet descompone una serie en tendencia, estacionalidad diaria y semanal y efecto de días festivos. Esta estructura coincide con el comportamiento de un parqueadero, cuya ocupación sube en horas laborales, baja en la noche y cambia en fines de semana y festivos. El servicio se ejecuta como un microservicio en Python, independiente de la API principal.

Función dentro del proyecto. El servicio estima el porcentaje de ocupación de cada sede para cada hora de las próximas 24 horas, con un intervalo de confianza. Atiende dos necesidades identificadas en el numeral 7. Al cliente le indica a qué hora encontrará cupo con mayor probabilidad, lo que reduce su tiempo de búsqueda. Al administrador le anticipa las horas pico para programar turnos de operadores, abrir espacios fuera de servicio o ajustar tarifas.

Integración con el sistema. La Figura 2 muestra el flujo de datos. Un proceso programado agrega cada noche los registros de la tabla registro_estacionamiento en una serie de ocupación por hora y por sede. El microservicio entrena el modelo con esa serie, genera el pronóstico y lo guarda en una nueva tabla prediccion_ocupacion (id_parqueadero, fecha_hora, ocupacion_estimada, limite_inferior, limite_superior). La API expone el endpoint GET /api/prediccion?idParqueadero=1 y la interfaz React muestra la predicción como una gráfica junto a la disponibilidad actual. El servicio solo lee datos operativos, por lo cual una falla del modelo no afecta los ingresos, las salidas ni los pagos.

Figura 2. ***Flujo de integración del servicio de predicción de ocupación***



Ejecución programada: reentrenamiento diario a las 2:00 a. m. y pronóstico de las próximas 24 horas

Fuente: elaboración propia.

Tabla 6. ***Ficha técnica del servicio de IA propuesto***

Elemento	**Definición**
Datos de entrada	Histórico de registros de estacionamiento (fecha de ingreso, fecha de salida, sede, tipo de vehículo) y calendario de festivos de Colombia.
Modelo	Prophet con estacionalidad diaria y semanal y efecto de festivos. Un modelo por sede.
Salida	Ocupación estimada por hora para las próximas 24 horas, con límites inferior y superior.
Frecuencia	Reentrenamiento diario en horario de baja demanda.
Métrica de evaluación	Error porcentual absoluto medio (MAPE) inferior al 20 % sobre las dos últimas semanas de datos reservadas para prueba (Hyndman y Athanasopoulos, 2021).
Requisito de datos	Mínimo ocho semanas de histórico por sede. Durante la etapa académica se usan datos simulados con el script seed.sql.

Fuente: elaboración propia.

Resultado esperado. El cliente decide la hora de llegada con información y evita circular en busca de cupo. El administrador pasa de reaccionar a la congestión a planear con anticipación los turnos y la capacidad. El sistema aprovecha los datos que ya registra en cada transacción, sin hardware adicional. Este servicio responde a la pregunta problema del numeral 8, pues añade a la disponibilidad en tiempo real una estimación de la disponibilidad futura.

14. Modelado de una red neuronal simple

El numeral 13 propuso un pronóstico de ocupación con Prophet para las próximas 24 horas. Ese modelo funciona bien para la tendencia del día, pero solo mira el calendario y el histórico. No sabe que en este momento la sede está casi llena ni cuántas reservas vienen en camino. Para cubrir ese vacío se modela una red neuronal pequeña, un perceptrón multicapa, que estima la ocupación de la hora siguiente con el estado actual del sistema. En esta fase no se entrena todavía. El objetivo es dejar definido el diseño para que el equipo lo implemente en la fase de desarrollo (numeral 17).

14.1 Propósito

La red responde una pregunta concreta: *¿qué porcentaje de la sede estará ocupado dentro de una hora?* Con esa cifra el sistema puede advertir al cliente antes de que confirme una reserva o salga hacia el parqueadero, y el operador puede prepararse para un pico. Los dos modelos se reparten el trabajo. Prophet dibuja la curva del día completo y se recalcula de noche. La red se consulta en línea, en cada solicitud, y corrige la estimación con lo que está pasando en ese instante.

14.2 Entradas y salida

Se eligieron seis entradas que la base de datos de Parkway ya registra, de modo que no hace falta hardware ni sensores. Todas se escalan al rango de 0 a 1 para que ninguna domine el cálculo por su magnitud. La Tabla 7 resume cada variable y su origen.

Tabla 7. ***Variables de entrada y salida de la red neuronal***

Variable	Descripción	Origen en la base de datos	Escala
x1 hora	Hora del día en que se hace la consulta (0 a 23).	Reloj del servidor (zona America/Bogota).	hora / 23
x2 día	Día de la semana, de lunes (0) a domingo (6).	Reloj del servidor.	día / 6
x3 festivo	Indica si el día es festivo en Colombia.	Calendario de festivos ya usado por Prophet.	0 o 1
x4 ocupación actual	Espacios ocupados sobre capacidad de la sede y tipo de vehículo.	espacio.estado = OCUPADO	0 a 1
x5 ocupación hace 7 días	Ocupación registrada a la misma hora la semana anterior.	registro_estacionamiento (histórico)	0 a 1
x6 reservas próximas	Reservas activas que inician en la próxima hora sobre la capacidad.	reserva.estado = ACTIVA	0 a 1

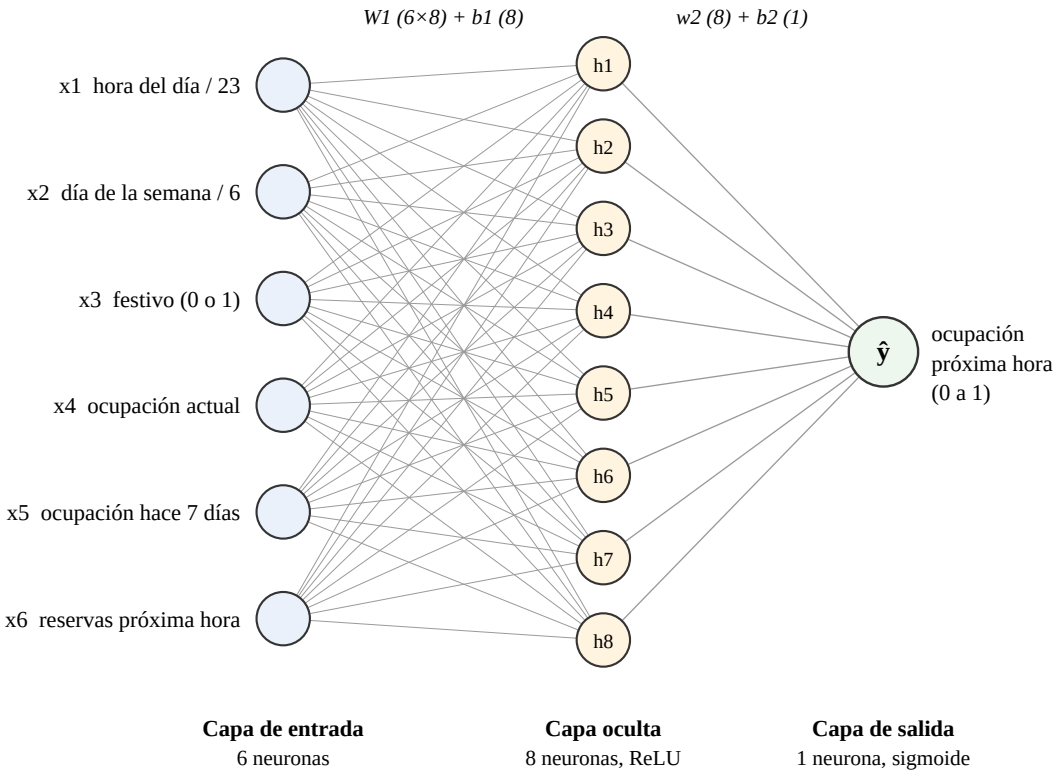
Variable	Descripción	Origen en la base de datos	Escala
ŷ salida	Ocupación estimada de la sede dentro de una hora.	Se guarda en prediccion_ocupacion.	0 a 1 (sigmoide)

Fuente: elaboración propia.

14.3 Arquitectura de la red

La Figura 3 muestra las tres capas. La capa de entrada recibe el vector x de seis valores. La capa oculta tiene ocho neuronas con activación ReLU, que deja pasar los valores positivos y anula los negativos; así la red puede captar relaciones no lineales, como el salto de ocupación que se da al final de la tarde. La salida es una sola neurona con función sigmoide, que entrega un número entre 0 y 1 y se lee directamente como porcentaje de ocupación.

Figura 3. ***Perceptrón multicapa para la ocupación de la próxima hora***



Fuente: elaboración propia.

El cálculo hacia adelante se resume en dos pasos: **h** = ReLU(W1·**x** + b1) y ŷ = σ(w2·**h** + b2). La red tiene 65 parámetros: 48 pesos y 8 sesgos en la capa oculta, más 8 pesos y 1 sesgo en la salida. Es un modelo deliberadamente pequeño. Una sede pequeña genera pocos miles de registros al mes y una red más grande se aprendería esos datos de memoria. Con 65 parámetros, además, cada integrante puede seguir el cálculo a mano y explicarlo en la

sustentación.

Un ejemplo ayuda a verlo. Un martes a las 5:00 p. m., sin festivo, la sede tiene 72 % de ocupación, hace una semana a esa hora tenía 85 % y hay 10 reservas para la próxima hora en una sede de 150 cupos. El vector queda x = (0,74; 0,17; 0; 0,72; 0,85; 0,07). Si la red ya entrenada devuelve $\hat{y}$ = 0,88, la API informa que se espera un 88 % de ocupación a las 6:00 p. m.

Tabla 8. ***Lineamientos para el entrenamiento en la fase de desarrollo***

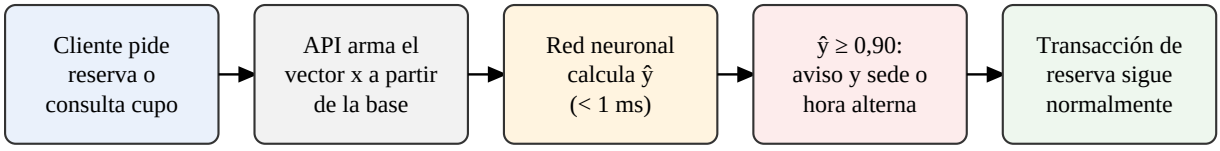
Aspecto	Decisión de diseño
Datos	Una fila por sede y por hora, armada desde registro_estacionamiento y reserva. Mientras no haya ocho semanas reales se usan los datos simulados del script seed.sql.
División	80 % de las semanas más antiguas para entrenar y 20 % más recientes para validar. No se mezclan al azar porque el orden en el tiempo importa.
Función de pérdida y optimizador	Error cuadrático medio (MSE) con el optimizador Adam (Kingma y Ba, 2015).
Métrica de aceptación	Error absoluto medio (MAE) de 8 puntos porcentuales o menos, y mejor que repetir la ocupación de hace una semana (modelo ingenuo).
Herramientas	Python con scikit-learn (MLPRegressor) o Keras en Google Colab. Los pesos se exportan a JSON y la API los evalúa en Node.js sin depender de Python en producción.

Fuente: elaboración propia.

14.4 Apoyo al proceso transaccional

La red no participa dentro de las transacciones de ingreso, salida o pago, y eso es intencional. El TransactionManager sigue abriendo y cerrando cada operación con BEGIN, COMMIT o ROLLBACK, igual que antes. Lo que cambia es la información con la que llega el usuario a esa transacción. La Figura 4 muestra el recorrido en una reserva.

Figura 4. ***Uso de la predicción dentro del flujo de reserva***



La predicción informa la decisión del usuario. Nunca bloquea ni revierte una transacción.

Fuente: elaboración propia.

- **Reserva (RF10).** Antes de confirmar, la API consulta la red. Si $\hat{y}$ es igual o mayor a 0,90, la pantalla avisa que la sede estará casi llena y sugiere otra sede o una hora con

menos demanda. El cliente decide; la reserva no se bloquea.

- **Operación (RF06 y RF07).** El operador ve en su panel la ocupación esperada y puede abrir un piso adicional o reforzar la portería antes del pico.
- **Administración.** Cada predicción se guarda en prediccion_ocupacion con el modelo que la produjo (MLP o PROPHET). Comparar lo estimado con lo que pasó permite medir el error mes a mes y decidir si hay que reentrenar.
- **Integridad.** Si la red falla o tarda, la API responde sin predicción y la transacción continúa. El servicio solo lee datos, así que no compromete la atomicidad ni el aislamiento definidos en el numeral 11.

15. Estado del arte

La búsqueda se hizo en IEEE Xplore, Springer, Taylor & Francis, arXiv y repositorios de universidades colombianas, con términos como *parking occupancy prediction*, *smart parking* y *software para parqueaderos*. Se priorizaron trabajos que predicen ocupación o que resuelven la gestión transaccional de un parqueadero, porque Parkway combina las dos cosas.

15.1 Antecedentes internacionales

Lin et al. (2017) revisaron más de cien soluciones de parqueo inteligente y las agruparon en tres frentes: recolección de información, sistemas de despliegue y difusión al usuario. Su conclusión es útil para este proyecto: buena parte de las propuestas depende de sensores por cupo, que son costosos de instalar y mantener.

En predicción, Zheng et al. (2015) compararon árboles de regresión, redes neuronales y regresión de vectores de soporte con datos de San Francisco y Melbourne. El árbol de regresión obtuvo el menor error, lo que muestra que un modelo sencillo bien alimentado puede competir con uno más complejo. Vlahogianni et al. (2016) usaron los sensores de Santander, en España, para predecir con redes neuronales la ocupación de los minutos siguientes y montaron el resultado en una aplicación web. Stolfi et al. (2017) probaron ajuste polinomial, series de Fourier, k-means y series de tiempo con los datos abiertos del concejo de Birmingham.

Con aprendizaje profundo, Camero et al. (2019) entrenaron redes recurrentes optimizadas con algoritmos evolutivos sobre 29 parqueaderos de Birmingham durante once semanas. Provoost et al. (2019) evaluaron en Arnhem, Países Bajos, una red neuronal prealimentada

y un bosque aleatorio para horizontes cortos; la red fue la mejor y ambos superaron la línea base ingenua. Ese resultado respalda la decisión del numeral 14 de usar un perceptrón y de compararlo siempre contra la ocupación de la semana anterior.

Del lado de la gestión, el programa SFpark de San Francisco ajustó tarifas según la demanda para mantener la ocupación entre 60 % y 80 % por cuadra (Pierce y Shoup, 2013). Ese rango sirve como referencia para los umbrales de alerta de Parkway.

15.2 Antecedentes en Colombia y Bogotá

En Bogotá, las Zonas de Parqueo Pago completaron 8.000 cupos en 13 localidades y ofrecen una aplicación con disponibilidad en tiempo real, reserva y pago electrónico (Alcaldía de Bogotá, 2023). Es la referencia local más cercana a lo que Parkway hace para una sede privada. En el ámbito académico, Solarte-Martínez et al. (2020) desarrollaron en Pereira un prototipo cliente-servidor para administrar parqueaderos con bitácora, auditoría y facturación. Cubillos García y Rodríguez Mora (2018) propusieron una arquitectura IoT para parqueaderos inteligentes en Bogotá basada en sensores y comunicación con la nube.

15.3 Aportes para Parkway

La Tabla 9 resume qué toma el proyecto de cada antecedente. El vacío que se identifica es claro: los trabajos de predicción se apoyan en sensores y no tratan la integridad de las operaciones, y los trabajos de gestión registran transacciones pero no anticipan la demanda. Parkway une los dos frentes. Predice con los mismos registros transaccionales que garantiza con propiedades ACID, y lo hace sin instalar un solo sensor.

Tabla 9. ***Síntesis del estado del arte y aportes al proyecto***

Trabajo	Contexto	Tecnología o enfoque	Aporte a Parkway
Lin et al. (2017)	Revisión mundial	Clasificación de soluciones de parqueo inteligente	Justifica evitar la dependencia de sensores.
Zheng et al. (2015)	San Francisco y Melbourne	Árbol de regresión, red neuronal y SVR	Un modelo pequeño puede ser suficiente.
Vlahogianni et al. (2016)	Santander, España	Redes neuronales y servicio web	Predicción de corto plazo publicada al usuario.
Stolfi et al. (2017)	Birmingham, Reino Unido	Polinomios, Fourier, k-means, series de tiempo	Validación cruzada con datos no vistos.
Camero et al. (2019)	Birmingham, 29 sedes	Redes recurrentes con optimización evolutiva	Opción futura cuando haya más datos.
Provoost et al. (2019)	Arnhem, Países Bajos	Red prealimentada y bosque aleatorio	Comparar siempre contra la línea base ingenua.
Pierce y Shoup (2013)	SFpark, San Francisco	Tarifa según demanda	Rango de 60 % a 80 % para fijar alertas.
Alcaldía de Bogotá (2023)	Zonas de Parqueo Pago	Aplicación con reserva y pago	Funciones esperadas por el usuario bogotano.
Solarte-Martínez et al. (2020)	Pereira	Software cliente-servidor con bitácora	Bitácora y auditoría de operaciones.
Cubillos García y Rodríguez Mora (2018)	Bogotá	Arquitectura IoT	Contraste: Parkway predice sin hardware.

Fuente: elaboración propia.

16. Presupuesto

El presupuesto cubre el semestre completo del proyecto. unas 18 semanas. y separa dos cosas, Por un lado. lo que el equipo pone en especie: su tiempo. sus computadores y su conexión, Por otro. el dinero que de verdad hay que pagar, Los valores están en pesos colombianos de noviembre de 2026, Cuando un costo está en dólares se convierte con una TRM de 3.182,99 pesos.

16.1 Recurso humano y tiempo

Cada integrante dedica cerca de 8 horas semanales fuera de clase, para un total de 432 horas. Como referencia se toma el salario promedio de un desarrollador junior en Colombia, $2.812.680 mensuales según Indeed (2026), dividido entre las 182 horas laborales del mes que fija la Ley 2101 de 2021 desde julio de 2026. El resultado es $15.454 por hora. Este valor no se paga: es el aporte del equipo, y se calcula para saber cuánto costaría el proyecto si se contratara.

Tabla 10. ***Recurso humano (aporte en especie)***

Integrante	Rol principal	Horas	Valor por hora	Valor total
Bryan Failach	Desarrollo backend y frontend, despliegue	144	$15.454	$2.225.417
Alejandro Castellanos	Pagos, pruebas y calidad	144	$15.454	$2.225.417
Brayan Bello	Datos y red neuronal	144	$15.454	$2.225.417
Total		**432**		**$6.676.251**

Fuente: elaboración propia con base en Indeed (2026).

16.2 Recursos tecnológicos, software y otros costos

Todo el software del proyecto es libre o tiene un plan gratuito suficiente para un proyecto académico, así que la línea de licencias queda en cero. El único servicio pago es el hosting del piloto: Render cobra 7 dólares al mes por el servicio web Starter y 6 por la base PostgreSQL Basic de 256 MB (Render, 2026). Durante el desarrollo se usa el plan gratuito y solo se paga en los tres meses de piloto. Los computadores se valoran por depreciación: un portátil de $3.000.000 con vida útil de cinco años, por los 4,5 meses del semestre.

Tabla 11. ***Recursos tecnológicos, software y otros costos***

Concepto	Detalle y justificación	Tipo	Valor
Computadores portátiles	3 equipos de $3.000.000, depreciación a 60 meses por 4,5 meses.	Propio	$675.000
Internet	25 % de un plan hogar de $80.000 al mes, 3 integrantes, 4,5 meses.	Propio	$270.000
Energía eléctrica	Portátil de 65 W por 144 h cada uno, a unos $950 por kWh.	Propio	$26.676
Software de desarrollo	VS Code, Node.js, TypeScript, PostgreSQL, React, Three.js, Git.	Gratuito	$0
Repositorio y colaboración	GitHub Free, Google Docs y PlantUML.	Gratuito	$0
Herramientas de IA	Python, scikit-learn, Keras y Google Colab (plan gratuito).	Gratuito	$0
Hosting del piloto	Render: 13 USD al mes por 3 meses.	Pago	$124.137
Impresión y materiales	Material de apoyo para la sustentación.	Pago	$30.000
Imprevistos	10 % de los gastos en efectivo, por variación de la TRM o un mes extra de hosting.	Pago	$15.414

Fuente: elaboración propia con base en Render (2026).

Tabla 12. ***Resumen del presupuesto***

Componente	**Valor**
Aporte en especie (recurso humano, equipos, internet y energía)	$7.647.927
Gastos en efectivo (hosting, impresión e imprevistos)	$169.550
Valor total estimado del proyecto	**$7.817.478**

Fuente: elaboración propia.

El equipo solo necesita reunir $169.550 en efectivo, unos $56.517 por persona. El resto del valor, $7.647.927, corresponde a recursos que ya tiene. El 85 % del costo total es tiempo de trabajo, algo normal en un proyecto de software donde casi todo el valor está en el desarrollo.

17. Cronograma

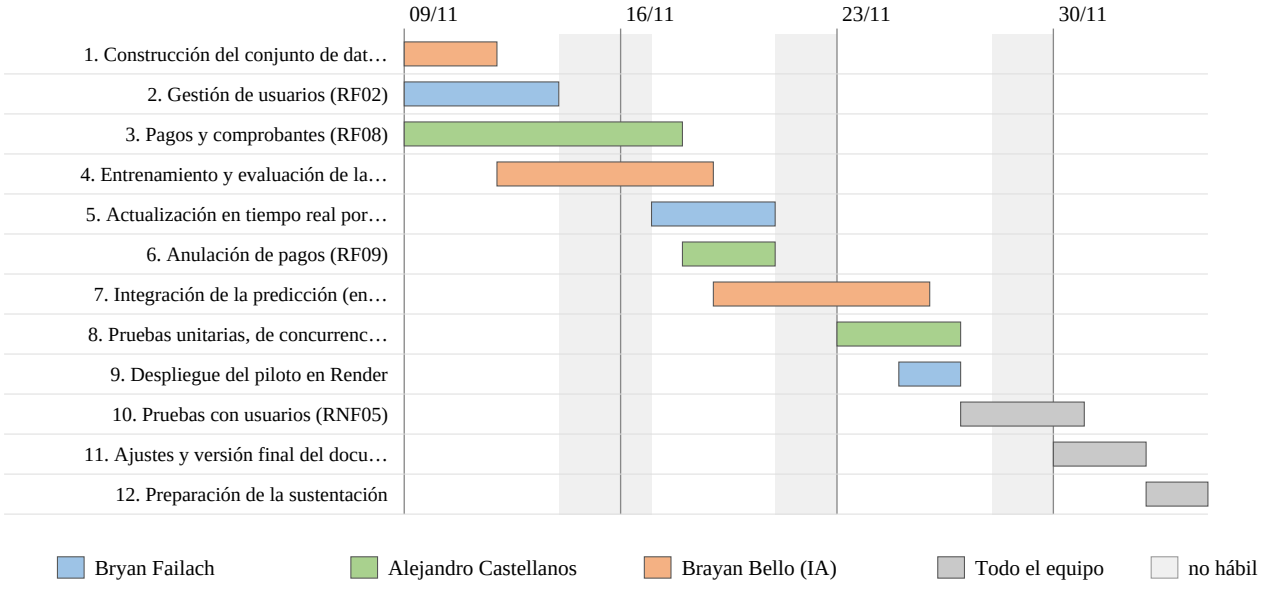
El cronograma cubre la fase de desarrollo que falta, del 9 de noviembre al 4 de diciembre de 2026. Son cuatro semanas con 19 días hábiles, porque el lunes 16 de noviembre es festivo. Las tareas se repartieron por afinidad: Bryan sigue con el núcleo de la aplicación y el despliegue, Alejandro con pagos y pruebas, y Brayan con los datos y la red neuronal. Las actividades del final son de todo el equipo. Varias tareas corren en paralelo, pero ninguna persona tiene más de dos al mismo tiempo.

Tabla 13. ***Cronograma de la fase de desarrollo***

N.º	Actividad	Responsable	Inicio	Fin	Días hábiles
1	Construcción del conjunto de datos de ocupación por hora	Brayan Bello	09/11/2026	11/11/2026	3
2	Gestión de usuarios (RF02)	Bryan Failach	09/11/2026	13/11/2026	5
3	Pagos y comprobantes (RF08)	Alejandro Castellanos	09/11/2026	17/11/2026	6
4	Entrenamiento y evaluación de la red neuronal	Brayan Bello y Bryan Failach	12/11/2026	18/11/2026	4
5	Actualización en tiempo real por WebSocket (RF12)	Bryan Failach	17/11/2026	20/11/2026	4
6	Anulación de pagos (RF09)	Alejandro Castellanos	18/11/2026	20/11/2026	3
7	Integración de la predicción (endpoint, tabla y vista)	Bryan Failach y Brayan Bello	19/11/2026	25/11/2026	5
8	Pruebas unitarias, de concurrencia y de carga (RNF02)	Alejandro Castellanos	23/11/2026	26/11/2026	4
9	Despliegue del piloto en Render	Bryan Failach	25/11/2026	26/11/2026	2
10	Pruebas con usuarios (RNF05)	Todo el equipo	27/11/2026	30/11/2026	2
11	Ajustes y versión final del documento maestro	Todo el equipo	30/11/2026	02/12/2026	3
12	Preparación de la sustentación	Todo el equipo	03/12/2026	04/12/2026	2

Fuente: elaboración propia.

Figura 5. ***Diagrama de Gantt de la fase de desarrollo***



Fuente: elaboración propia.

Las dependencias se respetan en el orden de las barras. La red no se entrena hasta tener el conjunto de datos (actividad 1), y la integración espera a que el modelo pase la métrica de aceptación de la Tabla 8. Las pruebas de carga con 100 usuarios simultáneos van después de cerrar pagos y anulaciones, y las pruebas con cinco usuarios reales se hacen sobre el piloto ya desplegado. Se dejaron dos días al final para ajustes, que funcionan como margen si alguna tarea se atrasa.

18. Referencias

Alcaldía de Bogotá. (2023, 21 de diciembre). *Zonas de Parqueo Pago completan 8.000 cupos de estacionamiento en Bogotá*. https://bogota.gov.co/mi-ciudad/movilidad/zonas-de-parqueo-pago-completan-8000-cupos-de-estacionamiento

Camero, A., Toutouh, J., Stolfi, D. H. y Alba, E. (2019). Evolutionary deep learning for car park occupancy prediction in smart cities. En R. Battiti, M. Brunato, I. Kotsireas y P. Pardalos (Eds.), *Learning and Intelligent Optimization. LION 12* (pp. 386-401). Springer. https://doi.org/10.1007/978-3-030-05348-2_32

Congreso de la República de Colombia. (2021). *Ley 2101 de 2021, por medio de la cual se reduce la jornada laboral semanal de manera gradual*.

Cubillos García, J. y Rodríguez Mora, J. (2018). *Arquitectura IoT para parqueaderos inteligentes en la ciudad de Bogotá* [Trabajo de grado, Universidad Católica de Colombia].

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fette, I. y Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Gray, J. y Reuter, A. (1992). *Transaction processing: Concepts and techniques*. Morgan Kaufmann.

Hyndman, R. J. y Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/

Indeed. (2026). *Sueldo de desarrollador junior en Colombia*. https://co.indeed.com

International Organization for Standardization. (2023). *ISO/IEC 25010:2023. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). Product quality model.*

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Kingma, D. P. y Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Laudon, K. C. y Laudon, J. P. (2020). *Management information systems: Managing the digital firm* (16.ª ed.). Pearson.

Lin, T., Rivano, H. y Le Mouël, F. (2017). A survey of smart parking solutions. *IEEE Transactions on Intelligent Transportation Systems, 18*(12), 3229-3253. https://doi.org/10.1109/TITS.2017.2685143

Meta Platforms. (s. f.). *React*. https://react.dev

Microsoft. (s. f.). *TypeScript documentation*. https://www.typescriptlang.org/docs/

OpenJS Foundation. (s. f.). *About Node.js*. https://nodejs.org/en/about

Pierce, G. y Shoup, D. (2013). Getting the prices right: An evaluation of pricing parking by demand in San Francisco. *Journal of the American Planning Association, 79*(1), 67-81. https://doi.org/10.1080/01944363.2013.787307

Provoost, J. C., Wismans, L. J. J., Van der Drift, S. J., Van Keulen, M. y Kamilaris, A. (2019). *Short term prediction of parking area states using real time data and machine learning techniques*. arXiv. https://arxiv.org/abs/1911.13178

Render. (2026). *Pricing*. https://render.com/pricing

Shoup, D. C. (2005). *The high cost of free parking*. American Planning Association.

Shoup, D. C. (2006). Cruising for parking. *Transport Policy, 13*(6), 479-486. https://doi.org/10.1016/j.tranpol.2006.05.005

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

Solarte-Martínez, G. R., Silva Castro, L. M. y Muñoz-Guerrero, L. E. (2020). Análisis, diseño y desarrollo de un prototipo de software para la administración de parqueaderos. *Ingeniería y Competitividad, 22*(1), 8752.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Stolfi, D. H., Alba, E. y Yao, X. (2017). Predicting car park occupancy rates in smart cities. En E. Alba, F. Chicano y G. Luque (Eds.), *Smart Cities. Smart-CT 2017* (pp. 107-117). Springer. https://doi.org/10.1007/978-3-319-59513-9_11

Taylor, S. J. y Letham, B. (2018). Forecasting at scale. *The American Statistician, 72*(1), 37-45. https://doi.org/10.1080/00031305.2017.1380080

The PostgreSQL Global Development Group. (2024a). *PostgreSQL documentation: Concurrency control*. https://www.postgresql.org/docs/current/mvcc.html

The PostgreSQL Global Development Group. (2024b). *PostgreSQL documentation: Partial indexes*. https://www.postgresql.org/docs/current/indexes-partial.html

Vlahogianni, E. I., Kepaptsoglou, K., Tsetsos, V. y Karlaftis, M. G. (2016). A real-time parking prediction system for smart cities. *Journal of Intelligent Transportation Systems, 20*(2), 192-204. https://doi.org/10.1080/15472450.2015.1037955

Zheng, Y., Rajasegarar, S. y Leckie, C. (2015). Parking availability prediction for sensor-enabled car parks in smart cities. En *2015 IEEE Tenth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)* (pp. 1-6). IEEE. https://doi.org/10.1109/ISSNIP.2015.7106902